

École Nationale des Sciences Appliquées d'AI Hoceima
Université Abdelmalek Essaadi

Compte Rendu de Travaux Pratiques

Module : Java EE (JEE)

Application de Gestion des Produits – MVC2

Annotations · Hibernate · JPA · Servlet unique · Base de données MySQL

Réalisé par	Nabila Es-Saidy
Encadrant	M. Cherradi
Année	2024 – 2025

ENSA AI Hoceima — Filière Génie Informatique

Introduction

Dans le cadre du module Java EE (JEE), ce compte rendu présente la réalisation d'une application web de gestion de produits suivant le patron architectural MVC2 (Model–View–Controller, variante 2). Cette version constitue une évolution significative par rapport à MVC1 : elle introduit les annotations Java EE pour la configuration des Servlets et des filtres, et remplace le stockage en mémoire par une persistance réelle en base de données MySQL via Hibernate/JPA.

L'application conserve les mêmes fonctionnalités que MVC1 — authentification, gestion des rôles, opérations CRUD sur les produits — tout en apportant une architecture plus moderne, plus concise et orientée vers la production.

Ce document décrit l'architecture technique MVC2, détaille les composants implémentés, met en évidence les différences clés avec MVC1, et illustre chaque fonctionnalité par des extraits de code commentés.

1. Présentation générale de l'application

1.1 Objectifs du projet

L'application GestionProduitsMVC2 est une application web Java EE qui évolue par rapport à MVC1 en s'appuyant sur les technologies suivantes :

- Java Servlets avec annotations `@WebServlet` — contrôleur HTTP centralisé
- JSP / JSTL — vues dynamiques (identiques à MVC1)
- Filtres Servlet avec annotations `@WebFilter` — sécurité sans web.xml
- Hibernate 5 / JPA — persistance en base de données MySQL
- Maven — gestion des dépendances et du build
- Apache Tomcat — serveur d'application

1.2 Fonctionnalités implémentées

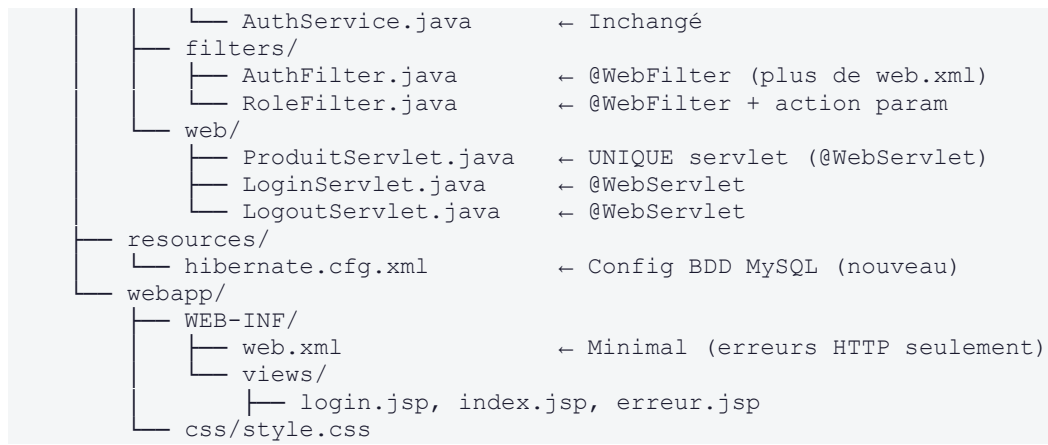
Fonctionnalité	Description	Rôle requis	URL MVC2
Authentification	Connexion par login/mot de passe avec session	Tous	/login
Déconnexion	Invalidation de la session	Tous	/logout
Liste des produits	Affichage de tous les produits (depuis MySQL)	Tous	/produit?action=list
Recherche par ID	Filtrage par identifiant	Tous	/produit?action=list&id=X
Ajout de produit	Création et persistance en BDD	ADMIN	POST /produit?action=add
Modification	Édition et mise à jour en BDD	ADMIN	POST /produit?action=update
Suppression	Suppression avec confirmation JavaScript	ADMIN	GET /produit?action=delete&id=X
Erreur 403	Accès refusé (rôle insuffisant)	Tous	—
Erreur 404/500	Pages d'erreur HTTP centralisées	Tous	—

1.3 Structure du projet

```

GestionProduitsMVC2/
├── pom.xml
├── src/main/
│   ├── java/ma/ensa/
│   │   ├── dao/
│   │   │   ├── HibernateUtil.java    ← sessionFactory (nouveau)
│   │   │   ├── ProduitDAO.java      ← Interface CRUD
│   │   │   └── ProduitDAOImpl.java   ← Impl. Hibernate (remplace ArrayList)
│   │   ├── models/
│   │   │   ├── Produit.java          ← @Entity @Table @Id (annotations JPA)
│   │   │   └── Utilisateur.java      ← Inchangé
│   │   └── services/
│   │       ├── ProduitService.java   ← Interface service
│   │       └── ProduitServiceImpl.java
└──

```



2. Différences clés entre MVC1 et MVC2

Cette section constitue le cœur du présent compte rendu. Elle met en évidence, aspect par aspect, les évolutions architecturales et techniques entre les deux versions du projet.

2.1 Tableau comparatif global

Aspect	MVC1 (ancien)	MVC2 (nouveau)
Configuration Servlets	web.xml (déclaratif XML)	@WebServlet (annotation Java)
Configuration Filtres	web.xml (filter-mapping XML)	@WebFilter (annotation Java)
Persistance des données	ArrayList en mémoire (volatile)	Hibernate + MySQL (persistante)
Modèle Produit	POJO simple, sans annotation	@Entity @Table @Id @Column (JPA)
Couche DAO	ProduitImpl (ArrayList)	ProduitDAOImpl (Session Hibernate)
Gestion session Hibernate	Absente	HibernateUtil.getSessionFactory()
Nombre de Servlets produits	5 Servlets séparées (List, Add, Edit, Update, Delete)	1 Servlet unique (ProduitServlet + action param)
Routage des actions	URL différente par action	Paramètre ?action= sur /produit
Dépendances Maven	javax.servlet-api + jstl	+ hibernate-core + mysql-connector
Fichier web.xml	Complet (servlets + filtres + erreurs)	Minimal (erreurs HTTP uniquement)
Package Java	dao/, models/, services/, filters/, web/	ma.ensa.dao/, ma.ensa.models/, etc. (packagété)
Données de démo	Initialisées en dur (ProduitImpl.init())	Insérées en BDD au démarrage ou manuellement

2.2 Configuration : annotations vs web.xml

C'est la différence la plus visible entre les deux versions. En MVC1, toute la configuration des Servlets et des filtres était centralisée dans web.xml. En MVC2, les annotations Java SE remplacent presque entièrement ce fichier.

MVC1 — Configuration dans web.xml

```
<!-- MVC1 : chaque servlet déclarée dans web.xml -->
```

```

<servlet>
  <servlet-name>ListProduitServlet</servlet-name>
  <servlet-class>web.ListProduitServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>ListProduitServlet</servlet-name>
  <url-pattern>/listProduits</url-pattern>
</servlet-mapping>

<!-- Idem pour AddProduitServlet, EditProduitServlet,
      UpdateProduitServlet, DeleteProduitServlet... -->

<!-- Filtres aussi déclarés en XML -->
<filter>
  <filter-name>AuthFilter</filter-name>
  <filter-class>filters.AuthFilter</filter-class>
</filter>
<filter-mapping>
  <filter-name>AuthFilter</filter-name>
  <url-pattern>/*</url-pattern>
</filter-mapping>

```

MVC2 — Annotations directement dans le code

```

// MVC2 : annotation @WebServlet sur la classe
@WebServlet("/produit")
public class ProduitServlet extends HttpServlet { ... }

@WebServlet("/login")
public class LoginServlet extends HttpServlet { ... }

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet { ... }

// Filtres configurés par annotation également
@WebFilter("/*")
public class AuthFilter implements Filter { ... }

@WebFilter("/produit")
public class RoleFilter implements Filter { ... }

```

MVC2 — web.xml réduit au minimum

```

<!-- web.xml MVC2 : uniquement les pages d'erreur HTTP -->
<web-app version="3.0">
  <display-name>GestionDesProduitsMVC2</display-name>

  <welcome-file-list>
    <welcome-file>index.jsp</welcome-file>
  </welcome-file-list>

  <error-page>
    <error-code>404</error-code>
    <location>/WEB-INF/views/erreur.jsp</location>
  </error-page>
  <error-page>
    <error-code>500</error-code>
    <location>/WEB-INF/views/erreur.jsp</location>
  </error-page>
</web-app>

```

2.3 Persistance : ArrayList → Hibernate + MySQL

C'est la différence architecturale la plus importante. En MVC1, les données étaient stockées dans une simple ArrayList en mémoire — elles disparaissaient à chaque redémarrage du serveur. En MVC2, Hibernate prend en charge la persistance vers une base de données MySQL réelle.

MVC1 — Stockage en mémoire (volatile)

```
// MVC1 : ProduitImpl.java – données dans une ArrayList
public class ProduitImpl implements ProduitDAO {
    private List<Produit> produits = new ArrayList<>();
    private Long compteurId = 0L;

    public void init() {
        // Données perdues à chaque redémarrage !
        addProduit(new Produit("Laptop Dell XPS", "Intel i7...", 12500.0));
        addProduit(new Produit("Souris Logitech MX", "Ergonomique", 450.0));
    }

    @Override
    public void addProduit(Produit p) {
        compteurId++;
        p.setIdProduit(compteurId); // ID manuel
        produits.add(p);
    }
}
```

MVC2 — Persistance avec Hibernate

```
// MVC2 : ProduitDAOImpl.java – opérations via Hibernate Session
public class ProduitDAOImpl implements ProduitDAO {

    private SessionFactory sf = HibernateUtil.getSessionFactory();

    @Override
    public void add(Produit p) {
        Transaction tx = null;
        try (Session s = sf.openSession()) {
            tx = s.beginTransaction();
            s.save(p); // INSERT INTO produits ...
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            throw new RuntimeException("Erreur ajout produit", e);
        }
    }

    @Override
    public List<Produit> getAll() {
        try (Session s = sf.openSession()) {
            // HQL – Hibernate Query Language
            return s.createQuery("FROM Produit ORDER BY idProduit").list();
        }
    }
}
```

2.4 Entité Produit : POJO → Entité JPA annotée

En MVC1, la classe `Produit` était un simple POJO sans aucune annotation. En MVC2, elle devient une entité JPA mappée sur la table `produits` de MySQL grâce aux annotations `@Entity`, `@Table`, `@Id`, `@GeneratedValue` et `@Column`.

MVC1 — POJO simple

```
// MVC1 : dao/Produit.java - POJO sans annotation
public class Produit {
    private Long idProduit;
    private String nom;
    private String description;
    private Double prix;
    // Getters / setters...
}
```

MVC2 — Entité JPA/Hibernate

```
// MVC2 : models/Produit.java - entité JPA annotée
@Entity
@Table(name = "produits")
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY) // AUTO_INCREMENT MySQL
    @Column(name = "id_produit")
    private Long idProduit;

    @Column(name = "nom", nullable = false, length = 150)
    private String nom;

    @Column(name = "description", length = 500)
    private String description;

    @Column(name = "prix", nullable = false)
    private Double prix;

    // Getters / setters...
}
```

2.5 Servlet unique vs Servlets multiples

En MVC1, chaque action CRUD disposait de sa propre Servlet (5 Servlets au total). En MVC2, une seule Servlet `ProduitServlet` centralise toutes les actions via un paramètre `action`.

MVC1 — 5 Servlets distinctes	MVC2 — 1 Servlet centrale (ProduitServlet)
ListProduitServlet → <code>/listProduits</code>	<code>/produit?action=list</code>
AddProduitServlet → <code>/addProduit</code>	<code>/produit?action=list&id=X</code>
EditProduitServlet → <code>/editProduit</code>	GET <code>/produit?action=edit&id=X</code>
UpdateProduitServlet → <code>/updateProduit</code>	POST <code>/produit?action=update</code>
DeleteProduitServlet → <code>/deleteProduit</code>	GET <code>/produit?action=delete&id=X</code>


```
// MVC2 : ProduitServlet.java – dispatch par paramètre action
@WebServlet("/produit")
public class ProduitServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res) {
        String action = req.getParameter("action");
        if (action == null) action = "list";

        switch (action) {
            case "list":    actionList(req, res);    break;
            case "edit":    actionEdit(req, res);    break;
            case "delete":  actionDelete(req, res);  break;
            default:        actionList(req, res);
        }
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res) {
        String action = req.getParameter("action");
        switch (action) {
            case "add":     actionAdd(req, res);     break;
            case "update":  actionUpdate(req, res);  break;
            default:        actionList(req, res);
        }
    }

    // ... méthodes actionList, actionEdit, actionAdd, actionUpdate,
    actionDelete
}
```

2.6 HibernateUtil — Gestionnaire de SessionFactory

MVC2 introduit une classe utilitaire HibernateUtil absente de MVC1. Elle garantit qu'une seule instance de SessionFactory est créée pour toute l'application (pattern Singleton), car l'initialisation d'une SessionFactory est une opération coûteuse.

```
// dao/HibernateUtil.java – Singleton SessionFactory
public class HibernateUtil {

    private static SessionFactory sessionFactory;

    public static synchronized SessionFactory getSessionFactory() {
        if (sessionFactory == null) {
            sessionFactory = new Configuration()
                .configure("hibernate.cfg.xml")
                .buildSessionFactory();
        }
        return sessionFactory;
    }

    public static void shutdown() {
        if (sessionFactory != null) sessionFactory.close();
    }
}
```

2.7 RoleFilter — Contrôle par paramètre action

En MVC1, le RoleFilter vérifiait l'URL (/addProduit, /deleteProduit, etc.) pour déterminer si l'action était réservée aux ADMIN. En MVC2, comme toutes les actions passent par la même URL /produit, le filtre inspecte désormais le paramètre action.

MVC1 — vérification par URL	MVC2 — vérification par paramètre action
<pre>boolean estActionAdmin = path.equals("/addProduit") path.equals("/deleteProduit") path.equals("/editProduit") path.equals("/updateProduit");</pre>	<pre>String action = request.getParameter("action"); boolean estActionAdmin = "add".equals(action) "delete".equals(action) "edit".equals(action) "update".equals(action);</pre>

3. Architecture MVC2

3.1 Principe du patron MVC2

Dans le patron MVC2, c'est le Contrôleur (Servlet) qui est le point d'entrée central : il reçoit les requêtes HTTP, orchestre le traitement, et délègue le rendu à la Vue (JSP). La JSP n'est plus jamais accédée directement — elle ne fait que présenter les données préparées par le contrôleur.

- Le navigateur envoie une requête HTTP vers /produit?action=...
- Le filtre AuthFilter vérifie l'authentification de la session
- Le filtre RoleFilter vérifie le rôle si l'action est sensible
- ProduitServlet dispatche selon le paramètre action
- La méthode action correspondante appelle le Service métier
- Le Service délègue à ProduitDAOImpl qui exécute des requêtes Hibernate/MySQL
- Le contrôleur place les données en attributs de requête et forward vers la JSP
- La JSP génère le HTML retourné au navigateur

3.2 Configuration Hibernate — hibernate.cfg.xml

Le fichier hibernate.cfg.xml, absent de MVC1, configure la connexion à MySQL et le comportement de Hibernate.

```
<!-- hibernate.cfg.xml -->
<hibernate-configuration>
  <session-factory>
    <!-- Connexion MySQL -->
    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>
    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/gestion_produits?useSSL=false
      &serverTimezone=UTC&allowPublicKeyRetrieval=true
    </property>
    <property name="hibernate.connection.username">appuser</property>
    <property name="hibernate.connection.password">apppass123</property>

    <!-- Dialecte et DDL -->
    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQL8Dialect
    </property>
    <!-- 'update' : crée/modifie la table automatiquement -->
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.connection.pool_size">5</property>

    <!-- Entité mappée -->
    <mapping class="ma.ensa.models.Produit"/>
  </session-factory>
</hibernate-configuration>
```

3.3 Dépendances Maven

MVC2 ajoute deux nouvelles dépendances par rapport à MVC1 : Hibernate Core et le connecteur MySQL JDBC.

```
<!-- pom.xml MVC2 – dépendances supplémentaires par rapport à MVC1 -->

<!-- Hibernate Core (ORM) -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>

<!-- Connecteur JDBC MySQL -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>

<!-- MVC1 avait uniquement : -->
<!-- javax.servlet-api (provided) + jstl -->
<!-- MVC2 conserve ces deux dépendances -->
```

4. Composants conservés depuis MVC1

Plusieurs composants de MVC1 sont réutilisés presque à l'identique dans MVC2. Cette section les résume rapidement pour mettre en évidence ce qui a réellement changé.

4.1 Modèle Utilisateur et AuthService

La classe Utilisateur (avec l'enum Role) et AuthService (authentification en mémoire avec HashMap) sont inchangés. L'authentification reste basée sur des comptes codés en dur — une évolution vers une table users en BDD pourrait constituer une amélioration future.

```
// Identique à MVC1 – comptes en mémoire
public class AuthService {
    private static AuthService instance;
    private final Map<String, Utilisateur> users = new HashMap<>();

    private AuthService() {
        users.put("admin", new Utilisateur("admin", "admin123", Role.ADMIN));
        users.put("user", new Utilisateur("user", "user123", Role.USER));
    }
}
```

4.2 LoginServlet et LogoutServlet

Ces deux Servlets ont le même comportement qu'en MVC1, à deux différences près : elles portent désormais l'annotation @WebServlet et redirigent vers /produit?action=list au lieu de /listProduits après une connexion réussie.

4.3 Vues JSP

Les vues login.jsp, index.jsp et erreur.jsp sont identiques à celles de MVC1. Les seuls ajustements concernent les URLs des actions dans les formulaires et les liens, qui pointent maintenant vers /produit?action=... au lieu de /addProduit, /editProduit, etc.

4.4 Filtre AuthFilter

Le filtre d'authentification AuthFilter est fonctionnellement identique. La seule différence est sa déclaration par annotation @WebFilter("/") au lieu d'un mapping XML dans web.xml.

5. Tests et résultats

5.1 Plan de tests fonctionnels

Test	Action	Résultat attendu	Différence vs MVC1
T01	Connexion admin/admin123	Session ADMIN, redirection vers /produit?action=list	URL de redirection différente
T02	Connexion user/user123	Session USER, mode lecture seule	Identique à MVC1
T03	Mauvais mot de passe	Message d'erreur, formulaire ré-affiché	Identique à MVC1
T04	Ajout d'un produit (ADMIN)	Produit inséré en BDD MySQL, liste mise à jour	Persistance réelle vs mémoire
T05	Ajout avec champ vide	Message d'erreur, produit non créé	Identique à MVC1
T06	Modification (ADMIN)	Formulaire pré-rempli, UPDATE en BDD	Persistance réelle vs mémoire
T07	Suppression (ADMIN)	DELETE en BDD, liste actualisée	Persistance réelle vs mémoire
T08	USER → /produit?action=add	Page 403 — accès refusé	Paramètre action vs URL distincte
T09	URL inconnue	Page 404 — introuvable	Identique à MVC1
T10	Déconnexion	Session invalidée, retour /login	Identique à MVC1
T11	Recherche par ID existant	Un seul produit affiché (depuis BDD)	Source : BDD vs ArrayList
T12	Recherche par ID inexistant	Message info, tous les produits affichés	Identique à MVC1

Conclusion

Ce projet MVC2 représente une évolution naturelle et significative de l'application MVC1. Les trois changements majeurs apportés sont :

- Le passage aux annotations Java EE (@WebServlet, @WebFilter) qui simplifie la configuration et réduit le fichier web.xml au strict minimum.
- L'intégration d'Hibernate/JPA qui remplace le stockage en mémoire par une persistance réelle dans MySQL — les données survivent aux redémarrages du serveur.
- La consolidation des Servlets en une seule ProduitServlet centrale avec dispatch par paramètre action — architecture plus maintenable et extensible.

Ces évolutions illustrent concrètement la progression vers une architecture Java EE moderne. Les prochaines étapes naturelles seraient l'utilisation de JPA EntityManager (niveau Jakarta EE), l'ajout d'un pool de connexions HikariCP, la persistance des utilisateurs en BDD, et l'intégration de Spring MVC ou Jakarta EE pour les projets en production.

En résumé : MVC1 → MVC2

web.xml complet → Annotations @WebServlet / @WebFilter
ArrayList en mémoire → Hibernate + MySQL (persistant)
5 Servlets séparées → 1 Servlet centrale (dispatch par action)
POJO simple → @Entity JPA avec @GeneratedValue
ID manuel (compteur) → AUTO_INCREMENT MySQL

Annexe A — Comptes de démonstration

Login	Mot de passe	Rôle	Permissions
admin	admin123	ADMIN	Lister, rechercher, ajouter, modifier, supprimer
user	user123	USER	Lister et rechercher uniquement (lecture seule)

Annexe B — Schéma de la base de données

Avec `hibernate.hbm2ddl.auto = update`, Hibernate génère automatiquement la table produits au premier démarrage si elle n'existe pas.

```
-- Table générée automatiquement par Hibernate
-- (équivalent SQL de l'entité @Entity Produit)

CREATE TABLE produits (
    id_produit BIGINT NOT NULL AUTO_INCREMENT,
    nom VARCHAR(150) NOT NULL,
    description VARCHAR(500),
    prix DOUBLE NOT NULL,
    PRIMARY KEY (id_produit)
);

-- Base de données cible :
-- URL : jdbc:mysql://localhost:3306/gestion_produits
-- Utilisateur : appuser / apppass123
```